

PROJECT: Real-Time Chat App using WebSockets and Serverless

Services: API Gateway (WebSocket API), Lambda, DynamoDB, Cognito

Goal: Create a scalable real-time messaging platform.

Skills: WebSocket management, identity federation, state handling.

Definition: A Real-Time Chat App using WebSockets and Serverless is an application that enables instant communication between users without page refresh. It uses Amazon API Gateway to maintain persistent connections for real-time data transfer. Backend processing is handled by AWS Lambda, which eliminates the need for managing servers. Messages and connection data are stored in Amazon DynamoDB for scalability and fast access. User authentication and identity management are managed using Amazon Cognito, making the system secure and scalable.

Scope of this project: The scope of a Real-Time Chat App using WebSockets and Serverless includes building a scalable messaging platform that supports instant communication between multiple users. It involves using Amazon API Gateway for real-time connections and AWS Lambda for backend processing without server management. The project can be extended to include user authentication with Amazon Cognito and data storage using Amazon DynamoDB. Additional features like group chats, message history, and notifications can be implemented. Overall, it demonstrates skills in serverless architecture, real-time systems, and cloud scalability.

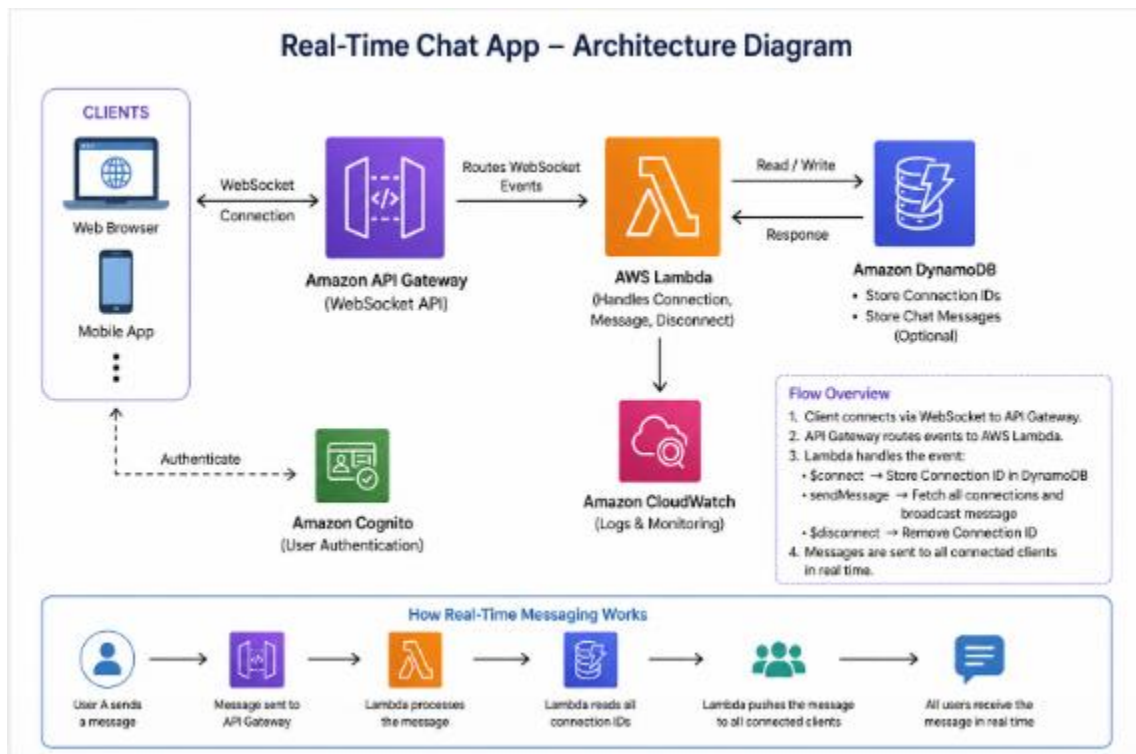
Methodologies:

The project uses an event-driven architecture where messages are processed in real time through Amazon API Gateway. Backend operations are handled using AWS Lambda, ensuring scalability without server management. Data storage and connection tracking are managed using Amazon DynamoDB. User authentication is implemented using Amazon Cognito for secure access. The development follows an Agile approach with iterative development, testing, and continuous deployment.

Services used in this Project:

- Amazon API Gateway – manages real-time WebSocket connections
- AWS Lambda – handles backend logic and message processing
- Amazon DynamoDB – stores connection IDs and chat data
- Amazon Cognito – provides user authentication and security

Architecture Diagram:



Implementation and Execution of Real-Time Chat App using WebSockets and Serverless

STEP 1: Create DynamoDB Tables

1. Connections Table
2. Messages Table

[DynamoDB](#) > [Tables](#)

Tables (2/2) [Info](#)

Last updated April 29, 2026, 01:36 (UTC+5:30)

[Actions](#) [Delete](#) [Create table](#)

Filter by tag Any tag key

Filter by tag value Any tag va...

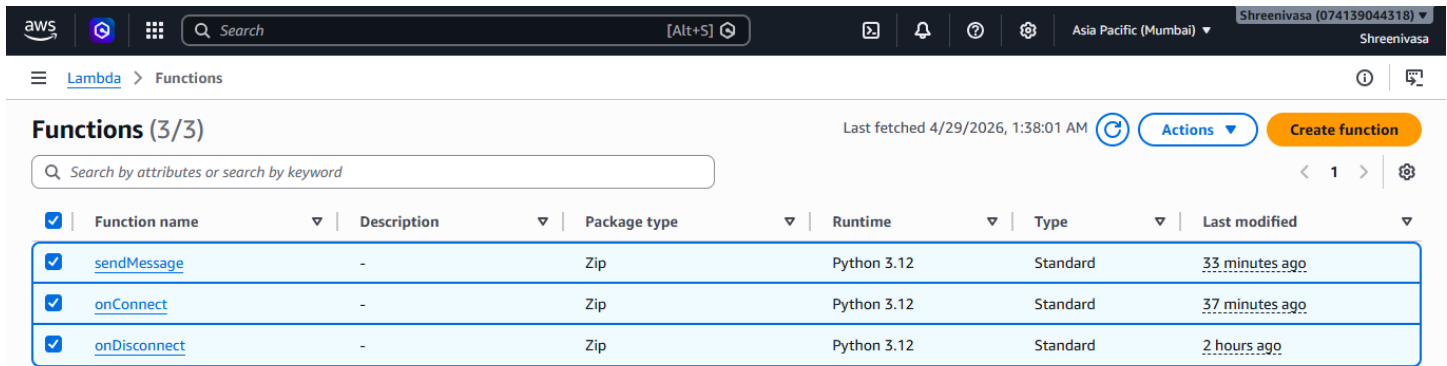
☒	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Filter
☒	Connections	Active	connectionId (S)	-	0	0	Off	
☒	Messages	Active	chatId (S)	timestamp (S)	0	0	Off	

STEP 2: Create Lambda Functions

Go to AWS Lambda → Create function

Created 3 functions:

1. onConnect
2. onDisconnect
3. sendMessage



The screenshot shows the AWS Lambda console with a list of three functions: `sendMessage`, `onConnect`, and `onDisconnect`. All functions are using Python 3.12 runtime and Standard architecture.

Function name	Description	Package type	Runtime	Type	Last modified
<code>sendMessage</code>	-	Zip	Python 3.12	Standard	33 minutes ago
<code>onConnect</code>	-	Zip	Python 3.12	Standard	37 minutes ago
<code>onDisconnect</code>	-	Zip	Python 3.12	Standard	2 hours ago

STEP 3: Create WebSocket API

Go to API Gateway → Create API → **WebSocket**

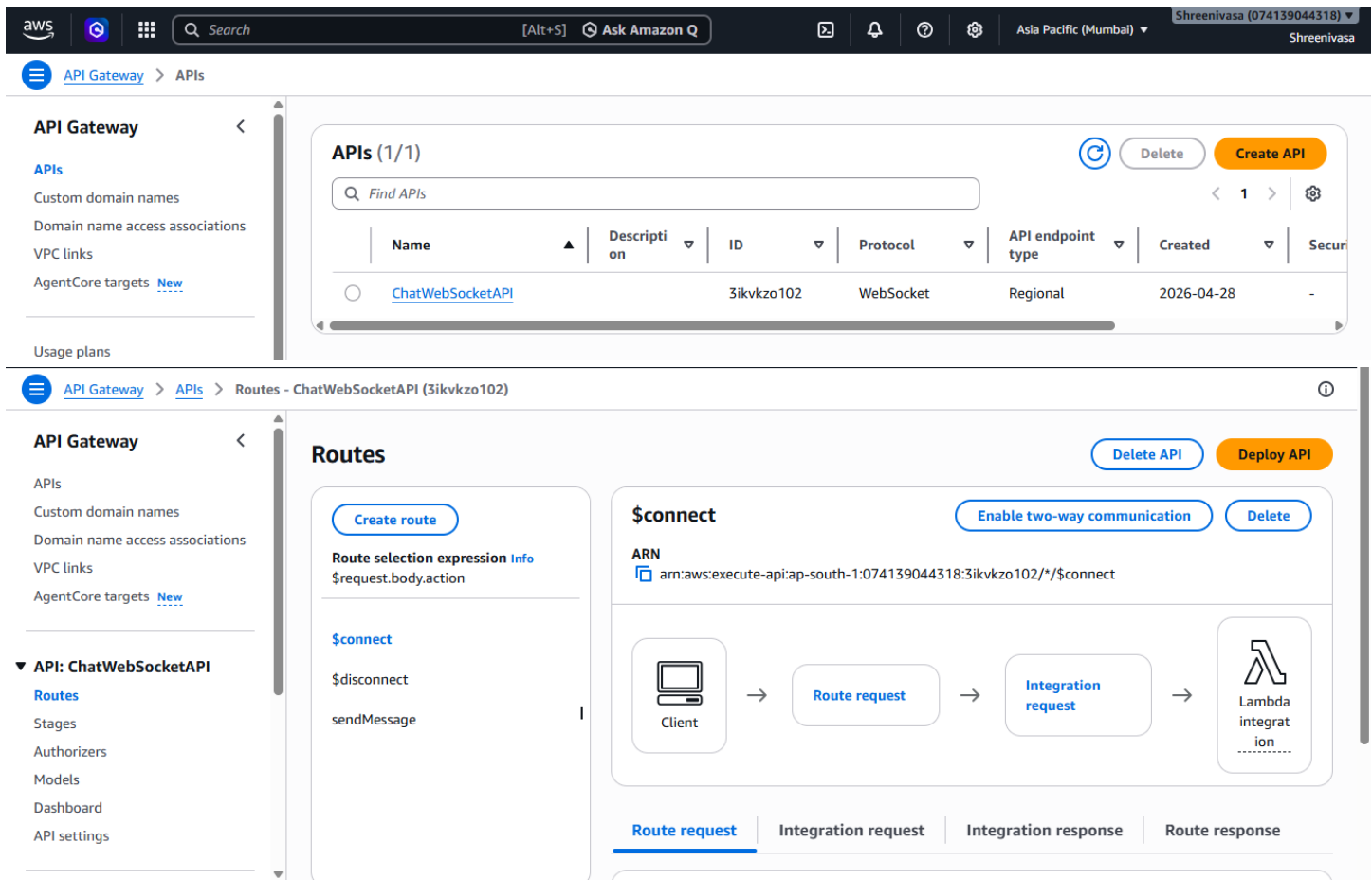
Name: ChatWebSocketApi

Configure Router names:

\$connect

\$disconnect

sendMessage



The screenshot shows the AWS API Gateway console with the configuration for a new WebSocket API named `ChatWebSocketAPI`. The API is in the `Routes` tab, showing the `$connect` route configuration.

APIs (1/1)

Name	Description	ID	Protocol	API endpoint type	Created	Security
<code>ChatWebSocketAPI</code>		3ikvkzo102	WebSocket	Regional	2026-04-28	-

Routes

\$connect

ARN: `arn:aws:execute-api:ap-south-1:074139044318:3ikvkzo102/*/$connect`

Route selection expression: `$request.body.action`

Routes: `$connect`, `$disconnect`, `sendMessage`

Integration: `Lambda integration`

Flow: Client → Route request → Integration request → Lambda integration

Tab: `Route request`

STEP 4: Set IAM Permissions for all the functions

Role: AmazonDynamoDBFullAccess

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

Roles

Policies

IAM users

IAM user groups

Identity providers

Account settings

Root access management

Temporary delegation requests

Access reports

Access Analyzer

Resource analysis

onConnect-role-wl86rkso

Creation date

April 28, 2026, 23:22 (UTC+05:30)

Last activity

36 minutes ago

ARN

arn:aws:iam::074139044318:role/service-role/onConnect-role-wl86rkso

Maximum session duration

1 hour

Permissions

Trust relationships

Tags

Last Accessed

Revoke sessions

Permissions policies (2)

Info

Simulate

Remove

Add permissions

You can attach up to 10 managed policies.

Search

Filter by Type

All types

< 1 >

Policy name

Type

Attached entities

AmazonDynamoDBFullAccess

AWS managed

3

AWSLambdaBasicExecutionRole-d...

Customer managed

1

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

Roles

Policies

IAM users

IAM user groups

Identity providers

Account settings

Root access management

Temporary delegation requests

Access reports

Access Analyzer

Resource analysis

onDisconnect-role-jtch9it1

Creation date

April 28, 2026, 23:27 (UTC+05:30)

Last activity

16 minutes ago

ARN

arn:aws:iam::074139044318:role/service-role/onDisconnect-role-jtch9it1

Maximum session duration

1 hour

Permissions

Trust relationships

Tags

Last Accessed

Revoke sessions

Permissions policies (2)

Info

Simulate

Remove

Add permissions

You can attach up to 10 managed policies.

Search

Filter by Type

All types

< 1 >

Policy name

Type

Attached entities

AmazonDynamoDBFullAccess

AWS managed

3

AWSLambdaBasicExecutionRole-3b0f2dd...

Customer managed

1

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

Roles

Policies

IAM users

IAM user groups

Identity providers

Account settings

Root access management

Temporary delegation requests

Access reports

Access Analyzer

Resource analysis

sendMessage-role-dv5yq52m

Creation date

April 28, 2026, 23:30 (UTC+05:30)

Last activity

34 minutes ago

ARN

arn:aws:iam::074139044318:role/service-role/sendMessage-role-dv5yq52m

Maximum session duration

1 hour

Permissions

Trust relationships

Tags

Last Accessed

Revoke sessions

Permissions policies (3)

Info

Simulate

Remove

Add permissions

You can attach up to 10 managed policies.

Search

Filter by Type

All types

< 1 >

Policy name

Type

Attached entities

AmazonDynamoDBFullAccess

AWS managed

3

AWSLambdaBasicExecutionRole-2278007...

Customer managed

1

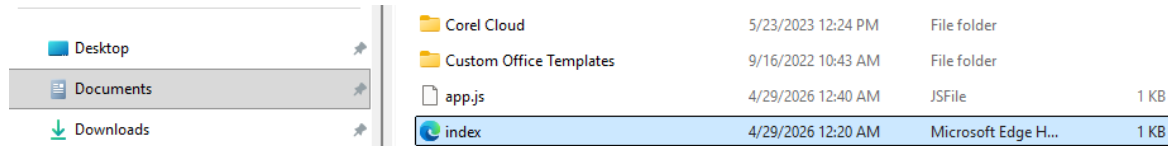
ManageConnectionsPolicy

Customer inline

0

STEP 6: Frontend (Simple HTML + JS)

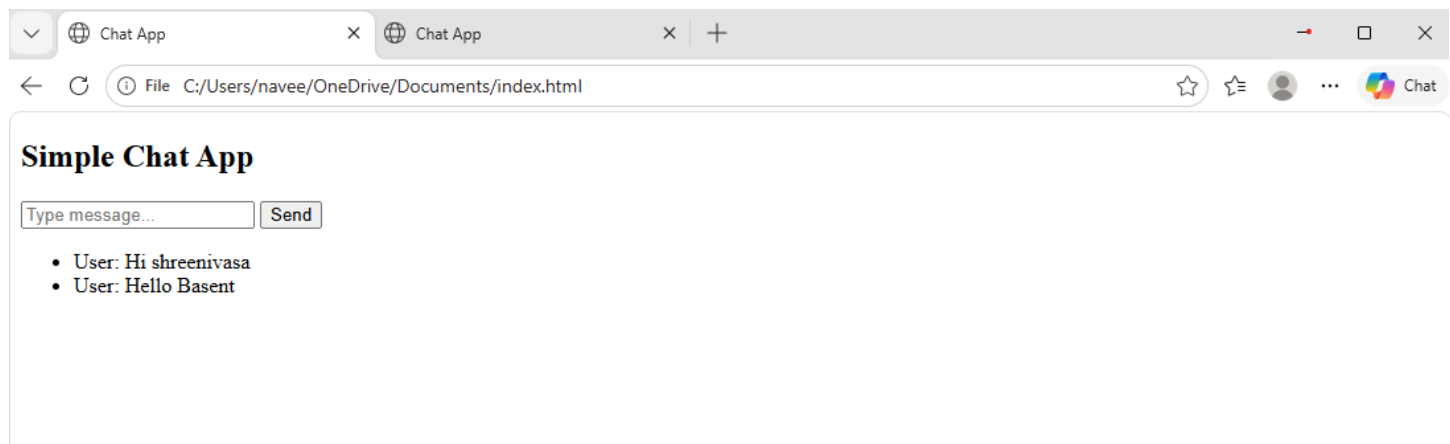
Created [index.html](#) and [app.js](#) files with the script I the local



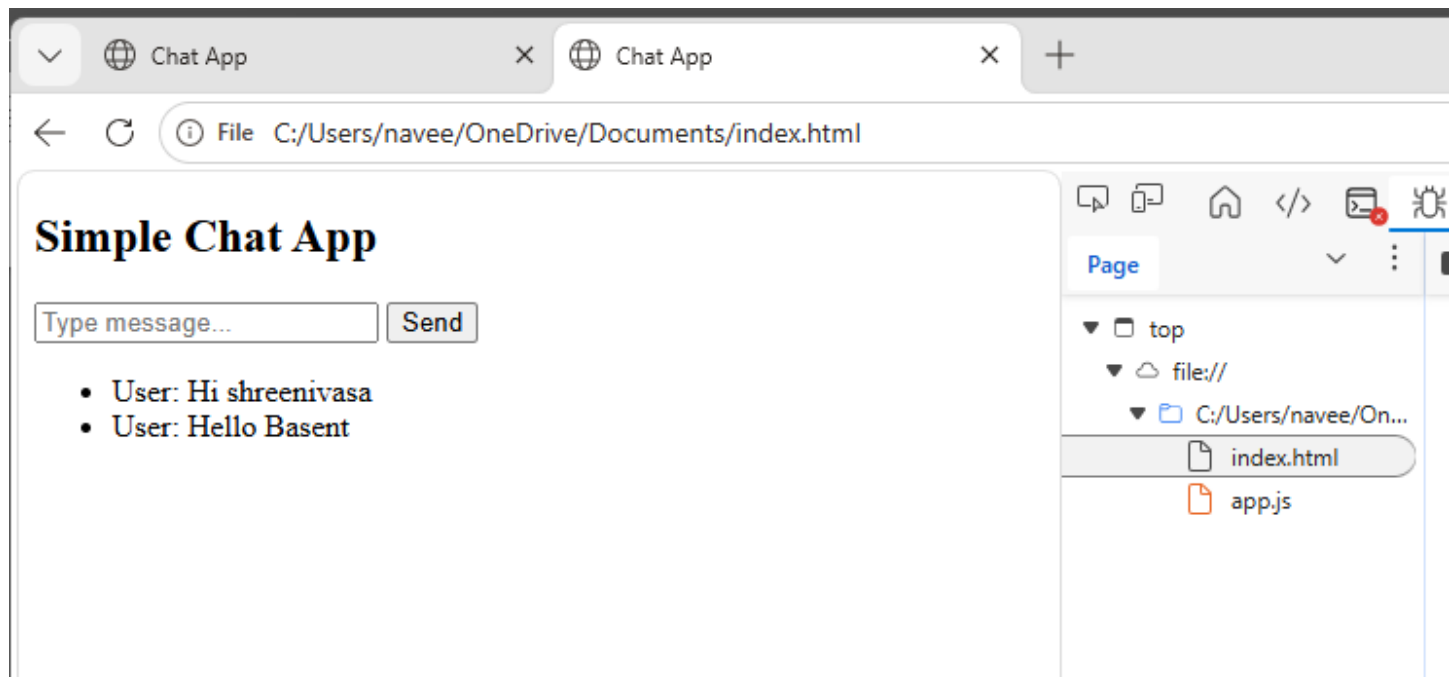
STEP 7: Deploy & Test

1. Open browser
2. Connect WebSocket
3. Send messages
4. Open multiple tabs → test real-time chat

Tab 1:



Tab 2:



Conclusion:

In conclusion, the Real-Time Chat App using WebSockets and Serverless is a scalable and efficient communication system that enables instant messaging between users without managing servers. It leverages Amazon API Gateway for real-time connectivity and AWS Lambda for backend processing. Data handling is managed through Amazon DynamoDB, ensuring fast and reliable storage. User authentication is secured using Amazon Cognito, making the application safe and scalable. Overall, this project demonstrates strong skills in cloud computing, real-time systems, and serverless architecture.